GUJARAT TECHNOLOGICAL UNIVERSITY

Navyug Vidyabhavan Trust

C. K. Pithawalla College of Engineering and Technology, Surat

A

Project Report

On

Handwritten Digit Recognition Using a Neural Network Built from Scratch

Prepared as a part of the requirements for the subject of

Python for Data Science (BE05000231)

B. E. III, Semester – V

(Computer Engineering)

Submitted by:

Sr. No.	Enrolment No	Name of Student	Division
1	240090107218	Hinal Tailor	B

Academic year (2026-27)

Problem Statement

Handwritten digit recognition is a classification problem in which a computer identifies a digit from a grayscale image. NEUROVISION solves this problem using a feed-forward neural network trained on MNIST. The model receives a 28×28 image (784 pixel values) and predicts one of the ten classes, 0–9.

Research / Idea Development

Objectives

- Understand the working of a neural network instead of using it as a black box.
- Implement forward propagation, ReLU, Softmax, cross-entropy, backpropagation and mini-batch gradient descent using NumPy.
- Train and evaluate the model on MNIST and visualize its performance.
- Demonstrate the trained model through a live handwritten-digit interface.

Dataset and Preparation

MNIST contains 70,000 handwritten digit images. Each image is 28×28 pixels and is flattened into 784 input features. Pixel values are normalized from 0–255 to 0–1. For this project, a stratified subset of 20,000 images was used.

Set	Images
Training	16,000
Validation	2,000
Final Test	2,000

Model Architecture

784 Input → 128 ReLU → 64 ReLU → 10 Softmax Output

- Hidden activation: ReLU
- Output activation: Softmax

- Loss: Cross-Entropy
- Learning: Backpropagation
- Optimization: Mini-Batch Gradient Descent

Methodology

1. Load and normalize MNIST data.
2. Create stratified training, validation and test sets.
3. Initialize weights and biases.
4. Perform forward propagation and calculate loss.
5. Calculate gradients using backpropagation and update parameters.
6. Train for 20 epochs and record loss/accuracy.
7. Evaluate once on the held-out test set and save results.
8. Load the trained weights in the live prediction application.

Implementation

Main training code

```
"""
NEUROVISION
Handwritten Digit Recognition Using a Neural Network Built from Scratch

PDS PBL - Beyond Syllabus Topic: Deep Learning with Neural Networks

Architecture:
    784 input neurons -> 128 ReLU -> 64 ReLU -> 10 Softmax

The neural-network learning logic is implemented manually with NumPy:
    Forward propagation
    ReLU activation
    Softmax activation
    Cross-entropy loss
    Backpropagation
    Mini-batch gradient descent

Dataset:
    MNIST handwritten digits (0-9)

Libraries:
    numpy, pandas, matplotlib, seaborn, scikit-learn, tensorflow
"""

# ============================================================
# 1. IMPORT LIBRARIES
# ============================================================

import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import fetch_openml
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, classification_report, accuracy_score

# Reproducibility
np.random.seed(42)

# ============================================================
# PROJECT OUTPUT DIRECTORY
```

```
# ============================================================
# All graphs, reports, predictions and trained weights are
# automatically saved here so they can be viewed later.
OUTPUT_DIR = "outputs"
os.makedirs(OUTPUT_DIR, exist_ok=True)

print("=" * 65)
print("NEUROVISION - HANDWRITTEN DIGIT RECOGNITION")
print("Neural Network Built from Scratch using NumPy")
print("=" * 65)


# ============================================================
# 2. LOAD MNIST DATASET
# ============================================================

print("\n[1/8] Loading MNIST dataset...")

try:
    mnist = fetch_openml("mnist_784", version=1, as_frame=False, parser="auto")
except TypeError:
    # Compatibility with older scikit-learn versions
    mnist = fetch_openml("mnist_784", version=1, as_frame=False)

X = mnist.data.astype(np.float32)
y = mnist.target.astype(np.int64)

print(f"Dataset shape: {X.shape}")
print(f"Number of classes: {len(np.unique(y))}")
print(f"Classes: {np.unique(y)}")


# ============================================================
# 3. PREPROCESSING
# ============================================================

print("\n[2/8] Preprocessing data...")

# Convert pixel values from 0-255 to 0-1
X = X / 255.0

# Use a stratified subset so the project remains practical on a normal laptop.
# Increase these values if your system is powerful.
MAX_SAMPLES = 20000

if len(X) > MAX_SAMPLES:
    X, _, y, _ = train_test_split(
        X,
```

```
        y,
        train_size=MAX_SAMPLES,
        stratify=y,
        random_state=42
    )

# Proper three-way split:
# Training   -> used to update model weights
# Validation -> monitored during training
# Test       -> used only once for final evaluation
X_train, X_temp, y_train, y_temp = train_test_split(
    X,
    y,
    test_size=0.20,
    stratify=y,
    random_state=42
)

X_val, X_test, y_val, y_test = train_test_split(
    X_temp,
    y_temp,
    test_size=0.50,
    stratify=y_temp,
    random_state=42
)

print(f"Training samples   : {len(X_train)}")
print(f"Validation samples : {len(X_val)}")
print(f"Testing samples    : {len(X_test)}")
print(f"Input features     : {X_train.shape[1]}")
print("Pixel range        : 0 to 1")



# ============================================================
# 4. VISUALIZE SAMPLE DIGITS
# ============================================================

print("\n[3/8] Displaying sample digits...")

fig, axes = plt.subplots(2, 5, figsize=(10, 4))

for i, ax in enumerate(axes.ravel()):
    ax.imshow(X_train[i].reshape(28, 28), cmap="gray")
    ax.set_title(f"Label: {y_train[i]}")
    ax.axis("off")

plt.suptitle("MNIST Sample Images", fontsize=15, fontweight="bold")
plt.tight_layout()
```

```
plt.savefig(os.path.join(OUTPUT_DIR, "01_sample_digits.png"), dpi=300, bbox_inches="tight")
plt.show()
plt.close()


# ============================================================
# 5. NEURAL NETWORK FROM SCRATCH
# ============================================================

class NeuralNetwork:
    """
    Feed-forward neural network implemented from scratch.

    Architecture:
        784 -> 128 -> 64 -> 10

    No high-level deep-learning model is used for training.
    """

    def __init__(self, input_size=784, hidden1=128, hidden2=64, output_size=10):
        # He initialization is suitable for ReLU layers.
        self.W1 = np.random.randn(input_size, hidden1).astype(np.float32) * np.sqrt(2 / input_size)
        self.b1 = np.zeros((1, hidden1), dtype=np.float32)

        self.W2 = np.random.randn(hidden1, hidden2).astype(np.float32) * np.sqrt(2 / hidden1)
        self.b2 = np.zeros((1, hidden2), dtype=np.float32)

        self.W3 = np.random.randn(hidden2, output_size).astype(np.float32) * np.sqrt(2 / hidden2)
        self.b3 = np.zeros((1, output_size), dtype=np.float32)

        self.loss_history = []
        self.accuracy_history = []

    # ---------- Activation functions ----------

    @staticmethod
    def relu(z):
        return np.maximum(0, z)

    @staticmethod
    def relu_derivative(z):
        return (z > 0).astype(np.float32)

    @staticmethod
    def softmax(z):
        # Subtract max for numerical stability.
        shifted = z - np.max(z, axis=1, keepdims=True)
        exp_values = np.exp(shifted)
```

```
    return exp_values / np.sum(exp_values, axis=1, keepdims=True)

# ---------- Forward propagation ----------

def forward(self, X):
    self.z1 = X @ self.W1 + self.b1
    self.a1 = self.relu(self.z1)

    self.z2 = self.a1 @ self.W2 + self.b2
    self.a2 = self.relu(self.z2)

    self.z3 = self.a2 @ self.W3 + self.b3
    self.a3 = self.softmax(self.z3)

    return self.a3

# ---------- Loss ----------

@staticmethod
def cross_entropy(y_true, y_prob):
    m = y_true.shape[0]
    probabilities = np.clip(y_prob[np.arange(m), y_true], 1e-12, 1.0)
    return -np.mean(np.log(probabilities))

# ---------- Backpropagation ----------

def backward(self, X, y):
    m = X.shape[0]

    # Output layer gradient
    dz3 = self.a3.copy()
    dz3[np.arange(m), y] -= 1
    dz3 /= m

    dW3 = self.a2.T @ dz3
    db3 = np.sum(dz3, axis=0, keepdims=True)

    # Hidden layer 2
    da2 = dz3 @ self.W3.T
    dz2 = da2 * self.relu_derivative(self.z2)

    dW2 = self.a1.T @ dz2
    db2 = np.sum(dz2, axis=0, keepdims=True)

    # Hidden layer 1
    da1 = dz2 @ self.W2.T
    dz1 = da1 * self.relu_derivative(self.z1)
```

```
    dW1 = X.T @ dz1
    db1 = np.sum(dz1, axis=0, keepdims=True)

    return dW1, db1, dW2, db2, dW3, db3

# ---------- Parameter update ----------

def update(self, gradients, learning_rate):
    dW1, db1, dW2, db2, dW3, db3 = gradients

    self.W1 -= learning_rate * dW1
    self.b1 -= learning_rate * db1

    self.W2 -= learning_rate * dW2
    self.b2 -= learning_rate * db2

    self.W3 -= learning_rate * dW3
    self.b3 -= learning_rate * db3

# ---------- Save / Load trained model ----------

def save_weights(self, filename):
    """Save all trained weights and biases."""
    np.savez(
        filename,
        W1=self.W1, b1=self.b1,
        W2=self.W2, b2=self.b2,
        W3=self.W3, b3=self.b3
    )

def load_weights(self, filename):
    """Load previously saved weights and biases."""
    data = np.load(filename)
    self.W1 = data["W1"]
    self.b1 = data["b1"]
    self.W2 = data["W2"]
    self.b2 = data["b2"]
    self.W3 = data["W3"]
    self.b3 = data["b3"]

# ---------- Prediction ----------

def predict_proba(self, X):
    return self.forward(X)

def predict(self, X):
    probabilities = self.predict_proba(X)
    return np.argmax(probabilities, axis=1)
```

```python
# ---------- Training ----------

def fit(
    self,
    X_train,
    y_train,
    X_val,
    y_val,
    epochs=20,
    batch_size=128,
    learning_rate=0.10
):
    n_samples = len(X_train)

    print("\n[4/8] Training neural network...")
    print("-" * 65)

    for epoch in range(epochs):
        # Shuffle training data each epoch
        indices = np.random.permutation(n_samples)
        X_shuffled = X_train[indices]
        y_shuffled = y_train[indices]

        # Mini-batch gradient descent
        for start in range(0, n_samples, batch_size):
            end = start + batch_size

            X_batch = X_shuffled[start:end]
            y_batch = y_shuffled[start:end]

            self.forward(X_batch)
            gradients = self.backward(X_batch, y_batch)
            self.update(gradients, learning_rate)

        # Track epoch metrics
        train_prob = self.forward(X_train)
        train_loss = self.cross_entropy(y_train, train_prob)
        train_pred = np.argmax(train_prob, axis=1)
        train_acc = np.mean(train_pred == y_train)

        val_prob = self.forward(X_val)
        val_loss = self.cross_entropy(y_val, val_prob)
        val_pred = np.argmax(val_prob, axis=1)
        val_acc = np.mean(val_pred == y_val)

        self.loss_history.append((train_loss, val_loss))
        self.accuracy_history.append((train_acc, val_acc))
```

```
        print(
            f"Epoch {epoch + 1:02d}/{epochs} | "
            f"Train Loss: {train_loss:.4f} | "
            f"Train Acc: {train_acc * 100:6.2f}% | "
            f"Val Loss: {val_loss:.4f} | "
            f"Val Acc: {val_acc * 100:6.2f}%"
        )


# ============================================================
# 6. CREATE + TRAIN MODEL
# ============================================================

model = NeuralNetwork(
    input_size=784,
    hidden1=128,
    hidden2=64,
    output_size=10
)

model.fit(
    X_train,
    y_train,
    X_val,
    y_val,
    epochs=20,
    batch_size=128,
    learning_rate=0.10
)

# Save trained parameters so future predictions can reuse the
# trained model without training from scratch.
weights_path = os.path.join(OUTPUT_DIR, "neurovision_weights.npz")
model.save_weights(weights_path)
print(f"\nSaved trained model weights to: {weights_path}")


# ============================================================
# 7. FINAL EVALUATION
# ============================================================

print("\n[5/8] Evaluating model on the FINAL TEST SET...")

test_probabilities = model.predict_proba(X_test)
y_pred = np.argmax(test_probabilities, axis=1)

accuracy = accuracy_score(y_test, y_pred)
```

```
print("\n" + "=" * 65)
print("FINAL MODEL PERFORMANCE")
print("=" * 65)
print(f"Final Test Accuracy : {accuracy * 100:.2f}%")
print(f"Correct       : {np.sum(y_pred == y_test)} / {len(y_test)}")
print(f"Incorrect     : {np.sum(y_pred != y_test)} / {len(y_test)}")
print("=" * 65)

print("\nClassification Report:")
report = classification_report(
    y_test,
    y_pred,
    digits=4,
    zero_division=0
)

print(report)

with open(os.path.join(OUTPUT_DIR, "classification_report.txt"), "w", encoding="utf-8") as f:
    f.write("NEUROVISION - CLASSIFICATION REPORT\n")
    f.write("=" * 60 + "\n")
    f.write(report)


# ============================================================
# 8. TRAINING GRAPHS
# ============================================================

print("\n[6/8] Creating training graphs...")

epochs_range = np.arange(1, len(model.loss_history) + 1)

train_losses = [x[0] for x in model.loss_history]
val_losses = [x[1] for x in model.loss_history]

train_accs = [x[0] * 100 for x in model.accuracy_history]
val_accs = [x[1] * 100 for x in model.accuracy_history]

# Loss graph
plt.figure(figsize=(9, 5))
plt.plot(epochs_range, train_losses, marker="o", label="Training Loss")
plt.plot(epochs_range, val_losses, marker="o", label="Validation Loss")
plt.title("Neural Network Learning Curve - Loss")
plt.xlabel("Epoch")
plt.ylabel("Cross-Entropy Loss")
plt.grid(alpha=0.3)
plt.legend()
```

```
plt.tight_layout()
plt.savefig(os.path.join(OUTPUT_DIR, "02_loss_curve.png"), dpi=300, bbox_inches="tight")
plt.show()
plt.close()

# Accuracy graph
plt.figure(figsize=(9, 5))
plt.plot(epochs_range, train_accs, marker="o", label="Training Accuracy")
plt.plot(epochs_range, val_accs, marker="o", label="Validation Accuracy")
plt.title("Neural Network Learning Curve - Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy (%)")
plt.grid(alpha=0.3)
plt.legend()
plt.tight_layout()
plt.savefig(os.path.join(OUTPUT_DIR, "03_accuracy_curve.png"), dpi=300, bbox_inches="tight")
plt.show()
plt.close()


# ============================================================
# 9. CONFUSION MATRIX
# ============================================================

print("\n[7/8] Creating confusion matrix...")

cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(9, 7))
sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=np.arange(10),
    yticklabels=np.arange(10)
)
plt.title("Confusion Matrix - MNIST Digit Recognition", fontsize=14, fontweight="bold")
plt.xlabel("Predicted Digit")
plt.ylabel("Actual Digit")
plt.tight_layout()
plt.savefig(os.path.join(OUTPUT_DIR, "04_confusion_matrix.png"), dpi=300, bbox_inches="tight")
plt.show()
plt.close()


# ============================================================
# 10. SAMPLE PREDICTIONS + CONFIDENCE
```

```
# ============================================================

print("\n[8/8] Showing model predictions...")

rng = np.random.default_rng(42)
sample_indices = rng.choice(len(X_test), size=12, replace=False)

fig, axes = plt.subplots(3, 4, figsize=(10, 8))

for ax, idx in zip(axes.ravel(), sample_indices):
    image = X_test[idx].reshape(28, 28)
    actual = y_test[idx]
    predicted = y_pred[idx]
    confidence = test_probabilities[idx, predicted] * 100

    ax.imshow(image, cmap="gray")
    ax.set_title(
        f"Actual: {actual} | Pred: {predicted}\n"
        f"Confidence: {confidence:.1f}%",
        fontsize=9
    )
    ax.axis("off")

plt.suptitle("Sample Predictions", fontsize=15, fontweight="bold")
plt.tight_layout()
plt.savefig(os.path.join(OUTPUT_DIR, "05_sample_predictions.png"), dpi=300, bbox_inches="tight")
plt.show()
plt.close()


# ============================================================
# 11. CONFIDENCE DISTRIBUTION FOR ONE PREDICTION
# ============================================================

# Pick a test example
demo_index = sample_indices[0]

probabilities = test_probabilities[demo_index]
predicted_digit = int(np.argmax(probabilities))
actual_digit = int(y_test[demo_index])

plt.figure(figsize=(10, 5))
bars = plt.bar(np.arange(10), probabilities * 100)
bars[predicted_digit].set_linewidth(3)

plt.xticks(np.arange(10))
plt.xlabel("Digit")
plt.ylabel("Probability (%)")
```

```
plt.title(
    f"Prediction Probability | Actual: {actual_digit} | "
    f"Predicted: {predicted_digit}"
)
plt.grid(axis="y", alpha=0.25)
plt.tight_layout()
plt.savefig(os.path.join(OUTPUT_DIR, "06_prediction_probability.png"), dpi=300, bbox_inches="tight")
plt.show()
plt.close()


# ============================================================
# 12. INTERACTIVE-STYLE CUSTOM IMAGE PREDICTION
# ============================================================

def predict_single_image(image_28x28):
    """
    Predict one 28x28 grayscale image.

    Input:
        image_28x28 -> values between 0 and 255 OR 0 and 1

    Returns:
        predicted digit and confidence
    """
    image = np.asarray(image_28x28, dtype=np.float32)

    if image.shape != (28, 28):
        raise ValueError("Image must have shape (28, 28).")

    if image.max() > 1:
        image = image / 255.0

    flattened = image.reshape(1, 784)

    probabilities = model.predict_proba(flattened)[0]
    prediction = int(np.argmax(probabilities))
    confidence = float(probabilities[prediction])

    return prediction, confidence, probabilities


# Example: use one existing test image as a demonstration
demo_image = X_test[demo_index].reshape(28, 28)

prediction, confidence, probabilities = predict_single_image(demo_image)

print("\n" + "=" * 65)
```

```python
print("NEUROVISION LIVE PREDICTION")
print("=" * 65)
print(f"Actual digit      : {actual_digit}")
print(f"Predicted digit   : {prediction}")
print(f"Confidence        : {confidence * 100:.2f}%")
print("=" * 65)

# Save prediction details
prediction_table = pd.DataFrame({
    "Digit": np.arange(10),
    "Probability": probabilities,
    "Probability_Percent": probabilities * 100
})
prediction_table.to_csv(
    os.path.join(OUTPUT_DIR, "07_demo_prediction_probabilities.csv"),
    index=False
)

# Save complete training history
history_table = pd.DataFrame({
    "Epoch": epochs_range,
    "Training_Loss": train_losses,
    "Validation_Loss": val_losses,
    "Training_Accuracy_Percent": train_accs,
    "Validation_Accuracy_Percent": val_accs
})
history_table.to_csv(
    os.path.join(OUTPUT_DIR, "08_training_history.csv"),
    index=False
)

# Save a compact project result summary
with open(os.path.join(OUTPUT_DIR, "09_project_results.txt"), "w", encoding="utf-8") as f:
    f.write("NEUROVISION - PROJECT RESULTS\n")
    f.write("=" * 60 + "\n")
    f.write("Dataset: MNIST Handwritten Digit Dataset\n")
    f.write("Original dataset size: 70,000 images\n")
    f.write("Project subset: 20,000 images\n")
    f.write(f"Training Samples: {len(X_train)}\n")
    f.write(f"Validation Samples: {len(X_val)}\n")
    f.write(f"Final Test Samples: {len(X_test)}\n")
    f.write(f"Final Test Accuracy: {accuracy * 100:.2f}%\n")
    f.write(f"Correct Predictions: {np.sum(y_pred == y_test)}\n")
    f.write(f"Incorrect Predictions: {np.sum(y_pred != y_test)}\n")
    f.write("Architecture: 784 -> 128 -> 64 -> 10\n")
    f.write("Activation: ReLU + Softmax\n")
    f.write("Loss: Cross-Entropy\n")
    f.write("Optimization: Mini-Batch Gradient Descent\n")
```

```
    f.write("Learning: Backpropagation\n")
    f.write("Final evaluation used a held-out test set not used for training.\n")



# ============================================================
# 13. PROJECT SUMMARY
# ============================================================

print("\nDATASET SPLIT")
print("-" * 65)
print(f"Training   : {len(X_train)} images")
print(f"Validation : {len(X_val)} images")
print(f"Final Test : {len(X_test)} images")
print("The final test set is not used for weight updates.")
print("-" * 65)

print("\nPROJECT SUMMARY")
print("-" * 65)
print("Topic            : Deep Learning with Neural Networks")
print("Application      : Handwritten Digit Recognition")
print("Architecture     : 784 -> 128 -> 64 -> 10")
print("Hidden Activation: ReLU")
print("Output Activation: Softmax")
print("Loss Function    : Cross-Entropy")
print("Optimizer Logic  : Mini-Batch Gradient Descent")
print("Learning Method  : Backpropagation")
print(f"Test Accuracy    : {accuracy * 100:.2f}%")
print("-" * 65)
print("Implementation completed successfully.")



# ============================================================
# 14. OUTPUT FILES CREATED
# ============================================================

print("\n" + "=" * 65)
print("ALL PROJECT OUTPUTS HAVE BEEN SAVED")
print("=" * 65)
for filename in sorted(os.listdir(OUTPUT_DIR)):
    print(f"  ✓ {filename}")

print("\nOpen the 'outputs' folder to view your saved graphs/results.")
print("You can run the program again; existing files will be replaced")
print("with the latest results.")
print("=" * 65)
```

Live prediction code

```
"""
NEUROVISION LIVE DIGIT DEMO

Run the training program first:
    python neurovision_mnist.py

Then:
    python live_digit_demo.py
"""

import os
import numpy as np
import tkinter as tk
from PIL import Image, ImageDraw

MODEL_FILE = os.path.join("outputs", "neurovision_weights.npz")


class NeuralNetwork:
    def __init__(self):
        self.W1 = np.zeros((784, 128), dtype=np.float32)
        self.b1 = np.zeros((1, 128), dtype=np.float32)
        self.W2 = np.zeros((128, 64), dtype=np.float32)
        self.b2 = np.zeros((1, 64), dtype=np.float32)
        self.W3 = np.zeros((64, 10), dtype=np.float32)
        self.b3 = np.zeros((1, 10), dtype=np.float32)

    @staticmethod
    def relu(z):
        return np.maximum(0, z)

    @staticmethod
    def softmax(z):
        z = z - np.max(z, axis=1, keepdims=True)
        exp_values = np.exp(z)
        return exp_values / np.sum(exp_values, axis=1, keepdims=True)

    def load_weights(self, filename):
        data = np.load(filename)
        self.W1, self.b1 = data["W1"], data["b1"]
        self.W2, self.b2 = data["W2"], data["b2"]
        self.W3, self.b3 = data["W3"], data["b3"]

    def predict_proba(self, X):
        a1 = self.relu(X @ self.W1 + self.b1)
        a2 = self.relu(a1 @ self.W2 + self.b2)
```

```
        return self.softmax(a2 @ self.W3 + self.b3)


class DigitApp:
    SIZE = 280

    def __init__(self, root):
        self.root = root
        self.root.title("NEUROVISION — Live Digit Recognition")
        self.root.geometry("760x560")
        self.root.resizable(False, False)

        if not os.path.exists(MODEL_FILE):
            raise FileNotFoundError(
                "Trained weights not found. Run neurovision_mnist.py first."
            )

        self.model = NeuralNetwork()
        self.model.load_weights(MODEL_FILE)

        self.image = Image.new("L", (self.SIZE, self.SIZE), 0)
        self.drawer = ImageDraw.Draw(self.image)
        self.last_x = self.last_y = None

        self.build_ui()

    def build_ui(self):
        tk.Label(
            self.root, text="NEUROVISION",
            font=("Segoe UI", 22, "bold")
        ).pack(pady=(12, 0))

        tk.Label(
            self.root,
            text="Draw a digit (0–9) and let the neural network recognize it",
            font=("Segoe UI", 11)
        ).pack(pady=(0, 12))

        main = tk.Frame(self.root)
        main.pack()

        left = tk.Frame(main)
        left.grid(row=0, column=0, padx=20)

        right = tk.Frame(main)
        right.grid(row=0, column=1, padx=20, sticky="n")

        self.canvas = tk.Canvas(
```

```
    left, width=self.SIZE, height=self.SIZE,
    bg="black", highlightthickness=2
)
self.canvas.pack()

self.canvas.bind("<Button-1>", self.start_draw)
self.canvas.bind("<B1-Motion>", self.draw)
self.canvas.bind("<ButtonRelease-1>", self.stop_draw)

tk.Button(
    left, text="CLEAR", command=self.clear,
    font=("Segoe UI", 11, "bold"), width=12
).pack(pady=12)

self.prediction = tk.Label(
    right, text="Prediction: —",
    font=("Segoe UI", 20, "bold")
)
self.prediction.pack(pady=(5, 20))

self.confidence = tk.Label(
    right, text="Confidence: —",
    font=("Segoe UI", 12)
)
self.confidence.pack(pady=(0, 15))

tk.Button(
    right, text="PREDICT DIGIT", command=self.predict,
    font=("Segoe UI", 12, "bold"), width=18, height=2
).pack(pady=(0, 18))

self.bars = []
self.values = []

for digit in range(10):
    row = tk.Frame(right)
    row.pack(fill="x", pady=2)

    tk.Label(row, text=str(digit), width=2).pack(side="left")

    bg = tk.Frame(row, width=180, height=16, bg="#dddddd")
    bg.pack(side="left", padx=5)
    bg.pack_propagate(False)

    bar = tk.Frame(bg, width=0, height=16, bg="#2563eb")
    bar.pack(side="left")

    value = tk.Label(row, text="0.0%", width=8, anchor="e")
```

```
        value.pack(side="left")

        self.bars.append(bar)
        self.values.append(value)

def start_draw(self, event):
    self.last_x, self.last_y = event.x, event.y
    self.draw(event)

def draw(self, event):
    if self.last_x is None:
        self.last_x, self.last_y = event.x, event.y

    width = 18

    self.canvas.create_line(
        self.last_x, self.last_y, event.x, event.y,
        fill="white", width=width,
        capstyle=tk.ROUND, smooth=True
    )

    self.drawer.line(
        [self.last_x, self.last_y, event.x, event.y],
        fill=255, width=width
    )

    r = width // 2
    self.drawer.ellipse(
        [event.x-r, event.y-r, event.x+r, event.y+r],
        fill=255
    )

    self.last_x, self.last_y = event.x, event.y

def stop_draw(self, event):
    self.last_x = self.last_y = None

def clear(self):
    self.canvas.delete("all")
    self.image = Image.new("L", (self.SIZE, self.SIZE), 0)
    self.drawer = ImageDraw.Draw(self.image)

    self.prediction.config(text="Prediction: —")
    self.confidence.config(text="Confidence: —")

    for bar, value in zip(self.bars, self.values):
        bar.config(width=0)
        value.config(text="0.0%")
```

```
    def preprocess(self):
        bbox = self.image.getbbox()

        if bbox is None:
            raise ValueError("Please draw a digit first.")

        cropped = self.image.crop(bbox)
        cropped.thumbnail((20, 20), Image.Resampling.LANCZOS)

        processed = Image.new("L", (28, 28), 0)
        x = (28 - cropped.width) // 2
        y = (28 - cropped.height) // 2
        processed.paste(cropped, (x, y))

        return (np.asarray(processed, dtype=np.float32) / 255.0).reshape(1, 784)

    def predict(self):
        try:
            x = self.preprocess()
        except ValueError as error:
            self.prediction.config(text=str(error))
            return

        probabilities = self.model.predict_proba(x)[0]
        digit = int(np.argmax(probabilities))
        confidence = float(probabilities[digit])

        self.prediction.config(text=f"Prediction: {digit}")
        self.confidence.config(
            text=f"Confidence: {confidence * 100:.2f}%"
        )

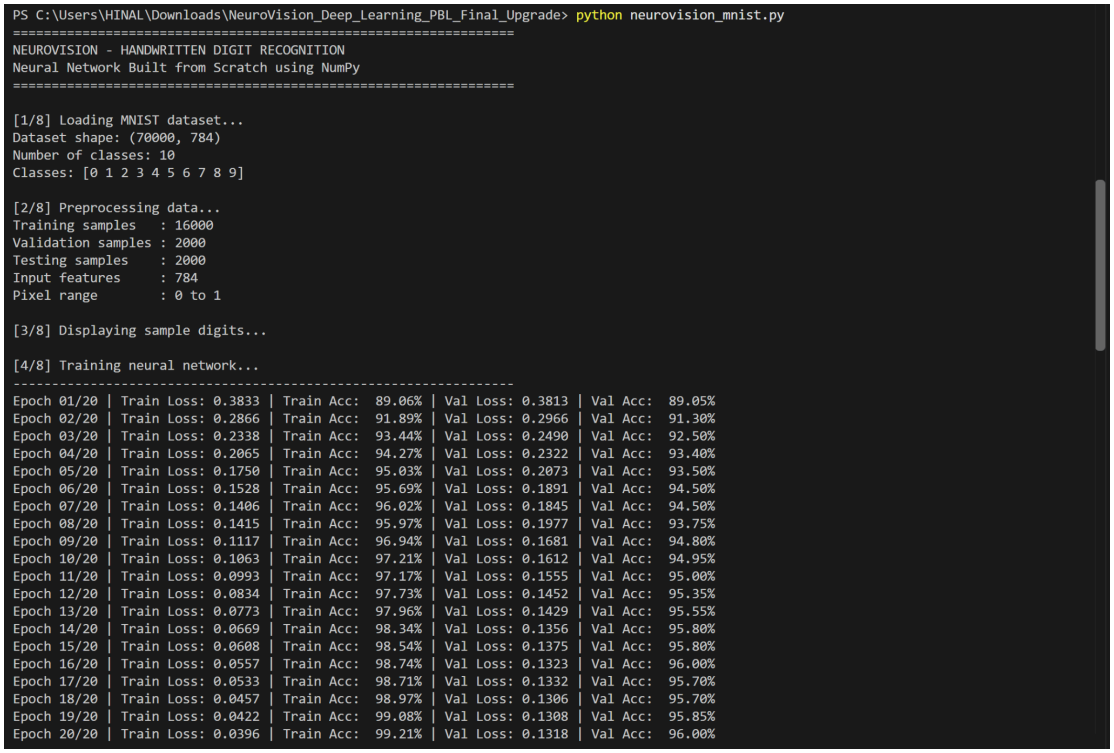
        for i, probability in enumerate(probabilities):
            self.bars[i].config(width=max(1, int(probability * 180)))
            self.values[i].config(text=f"{probability * 100:.1f}%")


if __name__ == "__main__":
    root = tk.Tk()
    DigitApp(root)
    root.mainloop()
```

Output and result

Metric	Result
Final Test Accuracy	96.00%
Correct Predictions	1,920 / 2,000
Incorrect Predictions	80 / 2,000
Epochs	20
Architecture	784 → 128 → 64 → 10

The model reached 96.00% accuracy on the held-out final test set. Training accuracy increased to 99.21%, while validation accuracy reached 96.00% by epoch 20. The loss curves show that the model learned progressively as training continued.



The training run completed 20 epochs, showing decreasing loss and increasing training and validation accuracy.

```
[5/8] Evaluating model on the FINAL TEST SET...

============================================================
FINAL MODEL PERFORMANCE
============================================================
Final Test Accuracy : 96.00%
Correct       : 1920 / 2000
Incorrect     : 80 / 2000
============================================================

Classification Report:
              precision    recall  f1-score   support

           0     0.9648    0.9746    0.9697       197
           1     0.9781    0.9911    0.9845       225
           2     0.9333    0.9800    0.9561       200
           3     0.9594    0.9265    0.9426       204
           4     0.9300    0.9538    0.9418       195
           5     0.9766    0.9278    0.9516       180
           6     0.9596    0.9645    0.9620       197
           7     0.9623    0.9808    0.9714       208
           8     0.9686    0.9487    0.9585       195
           9     0.9691    0.9447    0.9567       199

    accuracy                         0.9600      2000
   macro avg     0.9602    0.9593    0.9595      2000
weighted avg     0.9603    0.9600    0.9599      2000


[6/8] Creating training graphs...

[7/8] Creating confusion matrix...

[8/8] Showing model predictions...

============================================================
NEUROVISION LIVE PREDICTION
============================================================
Actual digit      : 2
Predicted digit   : 2
Confidence        : 99.94%
```

The held-out test set achieved 96.00% accuracy: 1,920 correct and 80 incorrect predictions. The classification report is also displayed.

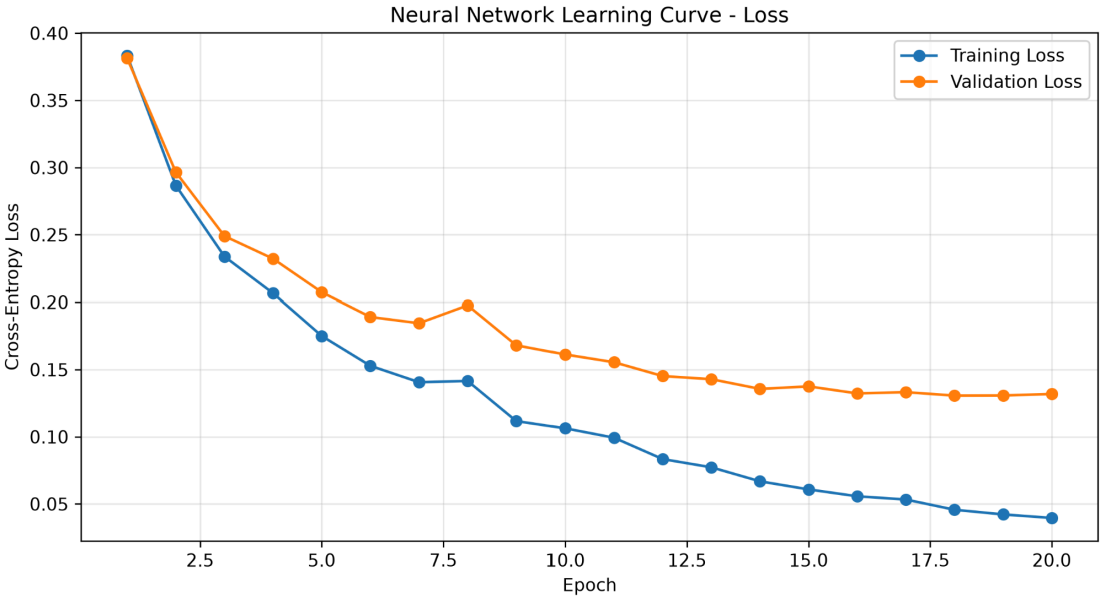
```
============================================================
NEUROVISION LIVE PREDICTION
============================================================
Actual digit      : 2
Predicted digit   : 2
Confidence        : 99.94%
============================================================

DATASET SPLIT
------------------------------------------------------------
Training   : 16000 images
Validation : 2000 images
Final Test : 2000 images
The final test set is not used for weight updates.
------------------------------------------------------------

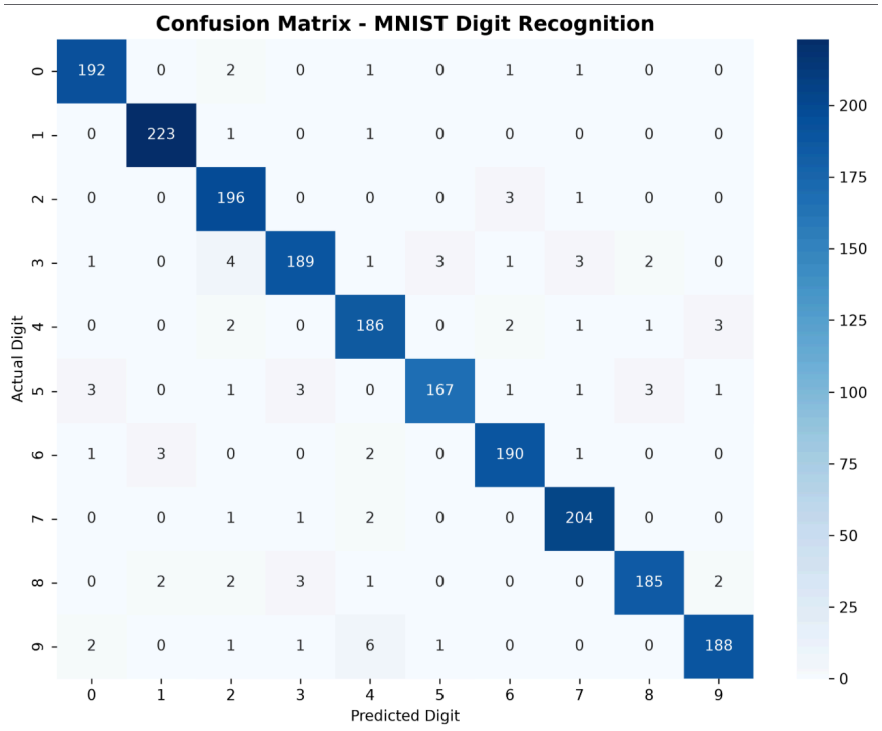
PROJECT SUMMARY
------------------------------------------------------------
Topic            : Deep Learning with Neural Networks
Application      : Handwritten Digit Recognition
Architecture     : 784 -> 128 -> 64 -> 10
Hidden Activation: ReLU
Output Activation: Softmax
Loss Function    : Cross-Entropy
Optimizer Logic  : Mini-Batch Gradient Descent
Learning Method  : Backpropagation
Test Accuracy    : 96.00%
------------------------------------------------------------
Implementation completed successfully.

============================================================
ALL PROJECT OUTPUTS HAVE BEEN SAVED
============================================================
  ✓ 01_sample_digits.png
  ✓ 02_loss_curve.png
  ✓ 03_accuracy_curve.png
  ✓ 04_confusion_matrix.png
  ✓ 05_sample_predictions.png
  ✓ 06_prediction_probability.png
  ✓ 07_demo_prediction_probabilities.csv
  ✓ 08_training_history.csv
```

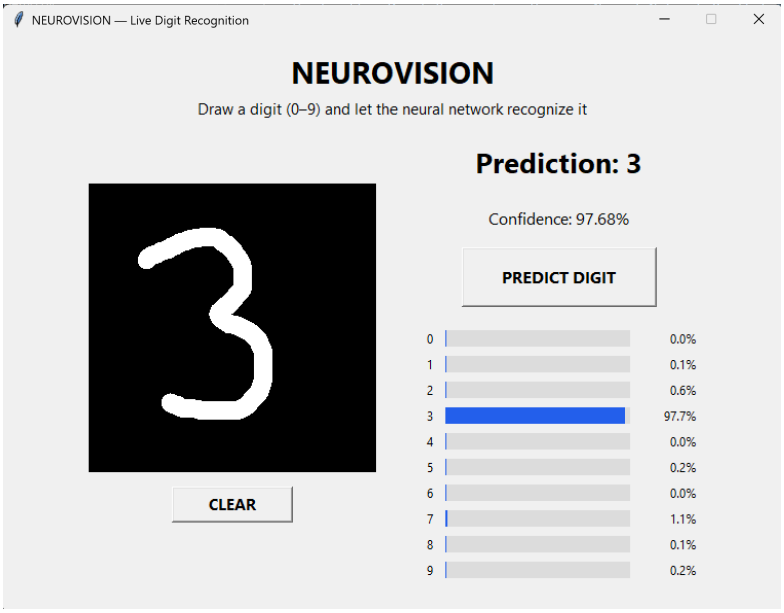
The console confirms the architecture, learning method, final accuracy, and successful saving of all project outputs.



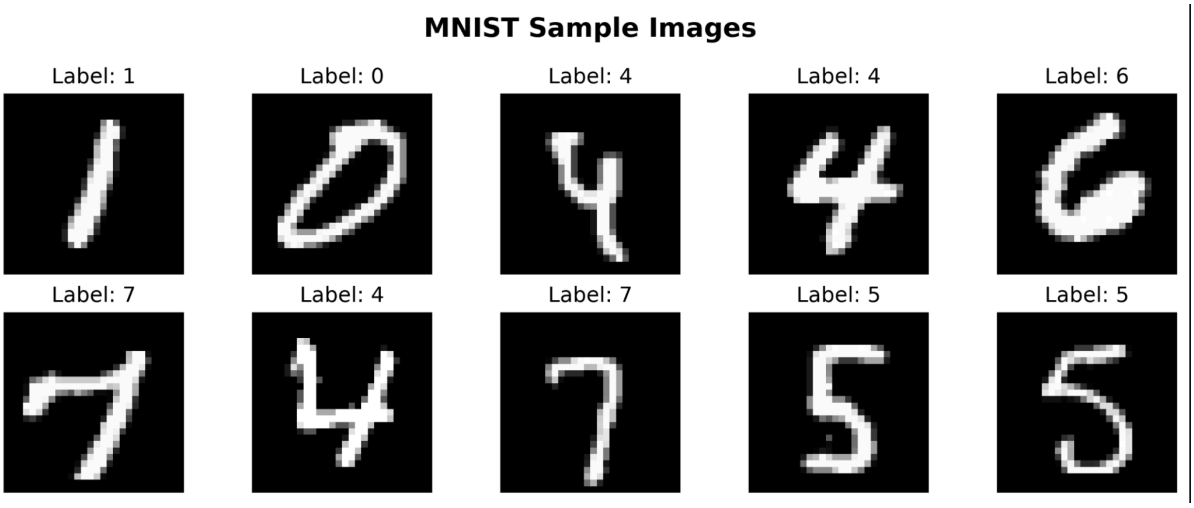
Training loss decreases steadily and validation loss also declines, indicating that the network is learning the digit patterns.



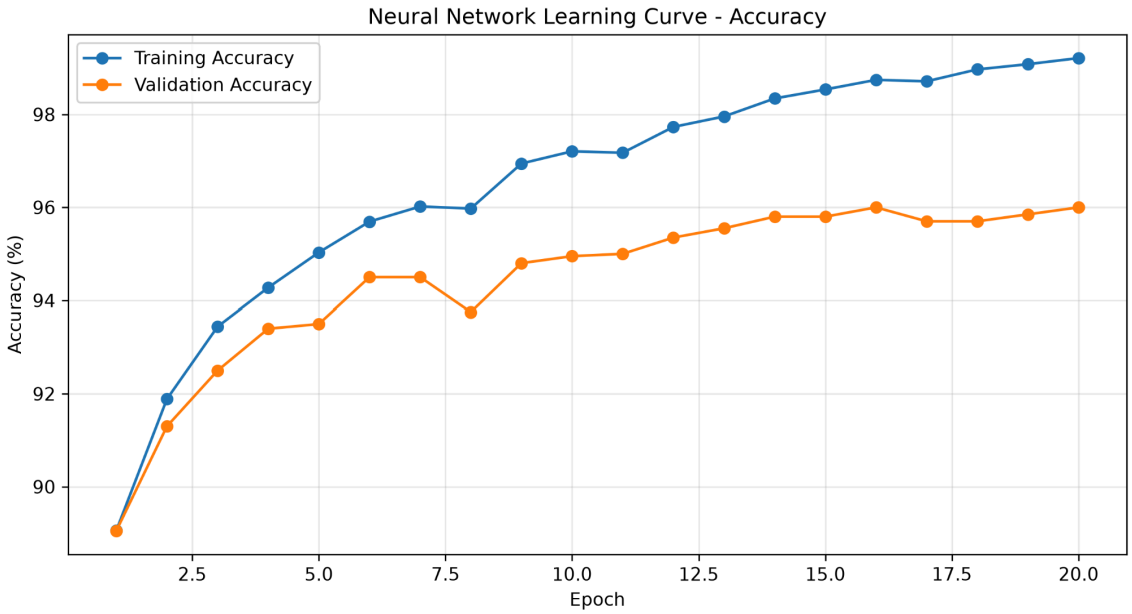
Most values lie on the diagonal, showing correct predictions. Off-diagonal values represent the relatively small number of misclassifications.



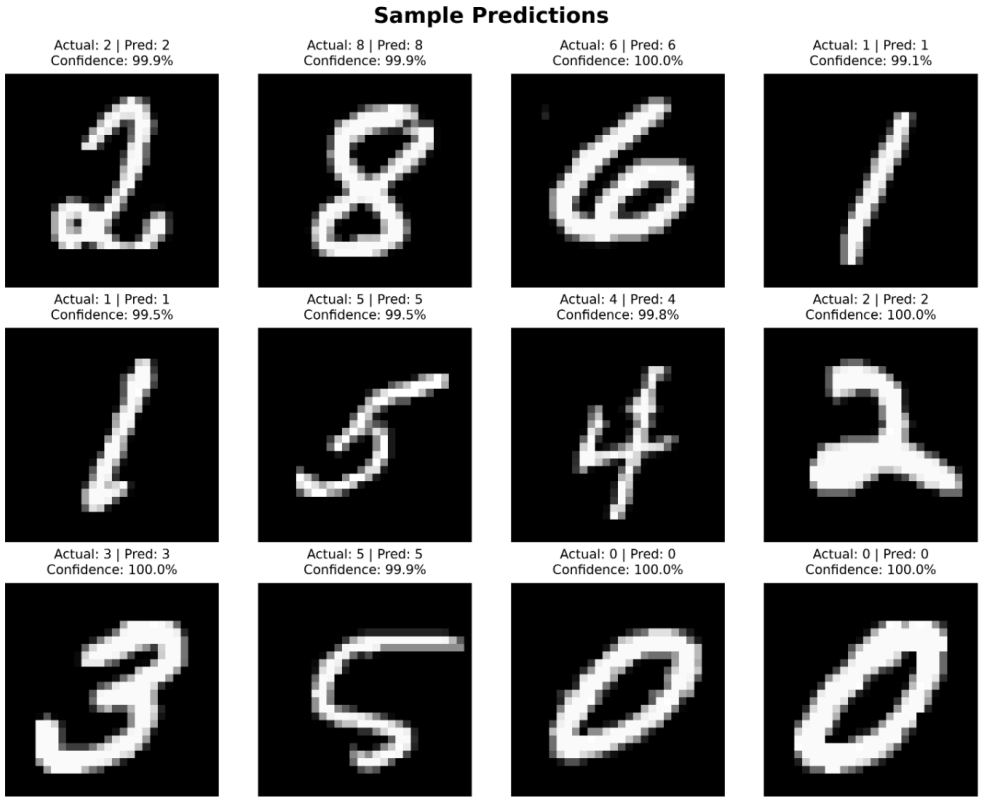
The live application recognized the drawn digit 3 with 97.68% confidence and displayed probabilities for all ten classes.



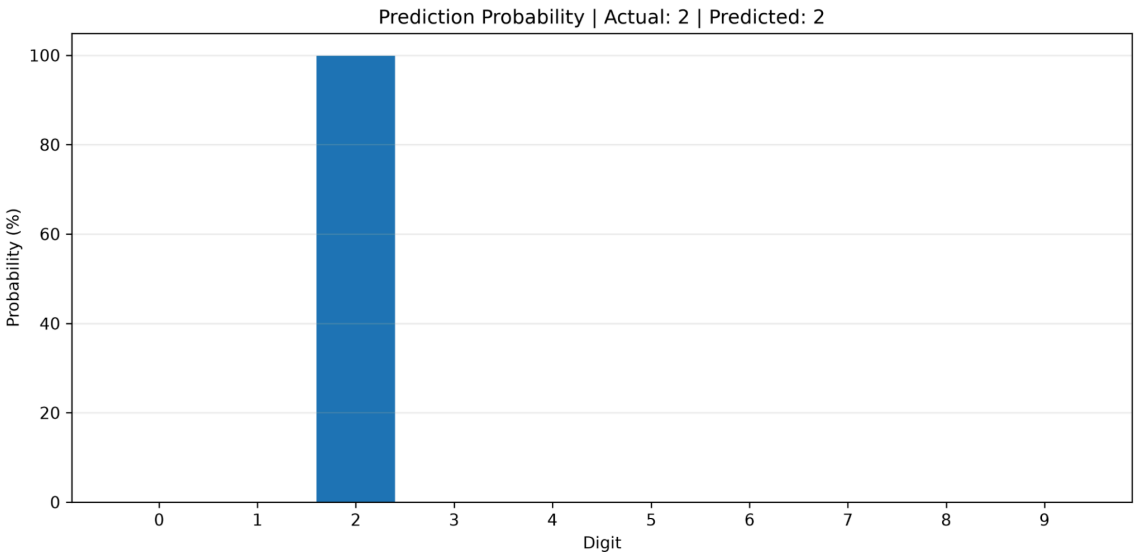
Ten sample MNIST images are displayed with their corresponding digit labels.



Training accuracy rises to 99.21% and validation accuracy reaches 96.00% by epoch 20.



Representative test images are shown with actual label, predicted label, and prediction confidence.



For the selected test image, class 2 has the dominant probability, supporting the predicted digit 2.

outputs > 07_demo_prediction_probabilities.csv

```
Digit,Probability,Probability_Percent
0,6.88734967858797e-07,6.88734967858797e-05
1,0.0005910157471280159,0.059101574712801586
2,0.9993689353419841,99.93689353419842
3,3.1799919187343134e-06,0.00031799919187343133
4,9.318040367194843e-09,9.318040367194843e-07
5,1.0403808021604594e-06,0.00010403808021604595
6,2.7242258201684908e-05,0.002724225820168490 7
7,1.487350155179849e-06,0.00014873501551798492
8,6.399012676422209e-06,0.0006399012676422209
9,1.8641255950064543e-09,1.8641255950064542e-07
```

The saved CSV records the probability produced for each digit class for the demo prediction.

outputs > 08_training_history.csv

```csv
Epoch,Training_Loss,Validation_Loss,Training_Accuracy_Percent,Validation_Accuracy_Percent
1,0.38330106198122194,0.38130797047368153,89.05625,89.05
2,0.2866476284180126,0.296551486774807,91.8875,91.3
3,0.2337884999536718,0.2490143923479351,93.44375000000001,92.5
4,0.20648983332458334,0.2321511067703729,94.27499999999999,93.4
5,0.17501259063303726,0.2073052218056857 2,95.03125,93.5
6,0.15282786662597425,0.189100524985873 3,95.69375,94.5
7,0.14062902122507007,0.18446912615222943,96.01875,94.5
8,0.1415443156063824,0.19769727333540646,95.975,93.75
9,0.11168651839197477,0.16806103709497955,96.94375,94.8
10,0.10628782976109101,0.16123539926122485,97.20625000000001,94.95
11,0.09929326654083939,0.15547392799537174,97.175,95.0
12,0.08339931779160799,0.14517898757028427,97.73125,95.35
13,0.07727508982114105,0.14286441188087007,97.95625,95.55
14,0.06685335260492191,0.1356348335146651,98.34375,95.8
15,0.06082746912584 9146,0.137509643807077,98.5375,95.8
16,0.05571543991326575,0.13228097968236477,98.74374999999999,96.0
17,0.05330603836246401,0.13321735292915002,98.7125,95.7
18,0.04568065790969858,0.13064036140429966,98.96875,95.7
19,0.04221251248491215,0.13075343659058716,99.08125,95.85000000000001
20,0.03958404378239 0374,0.13184576375360982,99.2125,96.0
```

The saved CSV records epoch-wise training loss, validation loss, training accuracy, and validation accuracy.

outputs > 09_project_results.txt

```
NEUROVISION - PROJECT RESULTS
============================================================
Dataset: MNIST Handwritten Digit Dataset
Original dataset size: 70,000 images
Project subset: 20,000 images
Training Samples: 16000
Validation Samples: 2000
Final Test Samples: 2000
Final Test Accuracy: 96.00%
Correct Predictions: 1920
Incorrect Predictions: 80
Architecture: 784 -> 128 -> 64 -> 10
Activation: ReLU + Softmax
Loss: Cross-Entropy
Optimization: Mini-Batch Gradient Descent
Learning: Backpropagation
Final evaluation used a held-out test set not used for training.

```

The saved text summary records the dataset size, split, final accuracy, architecture, loss, optimization, and learning method.

outputs > classification_report.txt

```
NEUROVISION - CLASSIFICATION REPORT
============================================================
              precision    recall  f1-score   support

           0     0.9648    0.9746    0.9697       197
           1     0.9781    0.9911    0.9845       225
           2     0.9333    0.9800    0.9561       200
           3     0.9594    0.9265    0.9426       204
           4     0.9300    0.9538    0.9418       195
           5     0.9766    0.9278    0.9516       180
           6     0.9596    0.9645    0.9620       197
           7     0.9623    0.9808    0.9714       208
           8     0.9686    0.9487    0.9585       195
           9     0.9691    0.9447    0.9567       199

    accuracy                         0.9600      2000
   macro avg     0.9602    0.9593    0.9595      2000
weighted avg     0.9603    0.9600    0.9599      2000

```

The saved classification report provides precision, recall, F1-score, and support for every digit class, along with macro and weighted averages.